



FPGA PRAKTIKUM SoSe 26

AUFGABE SIGNALGENERATOR

Fakultät 2
der Hochschule Furtwangen

im März 2026

Betreuer: Prof. Dr. Richard Spiegelberg
Autor: Marc Staiger

Inhaltsverzeichnis

1	Einleitung	1
2	Aufbau und Bedienung	2
2.1	Stromversorgung und Anschluss	3
2.2	Signalabnahme	3
2.3	Anzeige LEDs	4
2.4	Bedienung	6
3	Funktionsweise	9
3.1	Architektur im Überblick	9
3.2	Ein Takt, viele Geschwindigkeiten	9
3.3	Wellenformerzeugung	10
3.3.1	Rechteck	10
3.3.2	Dreieck	10
3.3.3	Sägezahn	10
3.3.4	Sinus (DDS)	11
3.4	Tasterentprellung	11
4	Aufgaben	12
4.1	Fehlende Codeteile rekonstruieren	12
4.2	Frequenzberechnung beim Sinusgenerator	15
4.3	Eigene Wellenform auf Modus X	15
4.4	Eigenen Ton für den Mystery-Modus	15
4.5	Zusatzaufgabe, LEDs im Takt der Melodie blinken lassen	16
4.6	Lösungen	16
	Abbildungsverzeichnis	I
5	Anhang	II

1 Einleitung

In dieser Aufgabe baust du deinen eigenen digitalen Signalgenerator direkt auf dem FPGA. Kein fertiger Funktionsgenerator, kein BlackBox Chip – du entwickelst selbst die komplette Logik, die später echte Wellenformen erzeugt. Das heißt: Du steuerst Frequenz, Amplitude und Signalform live über Taster und siehst auf dem Oszilloskop sofort, was deine Schaltung macht. Genau so, wie man es sich als Entwickler eigentlich immer wünscht: Knopf drücken, Signal ändert sich, und du weißt, dass es *deine* digitale Hardware ist, die das gerade tut.

Damit das klappt, kombinierst du mehrere richtig coole Bausteine: einen DDS basierten Sinusgenerator, sowie einen Rechteck und Sägezahn und Dreiecksgenerator mit einem dynamischen Taktteiler. Über LEDs bekommst du jederzeit Feedback, welcher Parameter aktiv ist und welche Wellenform gerade ausgegeben wird. Wenn du alles sauber programmierst, kannst du per Knopfdruck zwischen Sinus, Dreieck, Rechteck und Sägezahn umschalten und die Parameter in Echtzeit verändern – ohne irgendeine Software, komplett in Hardware.

Kurz gesagt: Du baust dir einen kleinen, aber vollwertigen digitalen Funktionsgenerator – komplett selbst programmiert, komplett auf dem FPGA, und du siehst dein Ergebnis direkt auf dem Oszilloskop. Klingt gut. Dann los.

2 Aufbau und Bedienung

Um das Projekt umzusetzen, erhältst du eine speziell entwickelte Erweiterungsplatine für dein FPGA Board. Bevor du sie anschließt, solltest du dir unbedingt den beigefügten Schaltplan ansehen und verstehen, welche Komponenten auf der Platine vorhanden sind und wie sie zusammenarbeiten. Wenn du nachvollziehen kannst, wie das Board aufgebaut ist, kannst du es vorsichtig auf dein FPGA aufstecken. Bitte gehe dabei sehr behutsam vor, die Lötstellen sind empfindlich und lassen sich bei unsachgemäßer Handhabung leicht beschädigen.

Behandle das Board insgesamt mit Sorgfalt. Jede Platine kostet rund 80 Euro, und die Herstellung erfordert fast zwei Stunden Arbeitszeit. Wenn du also vermeiden möchtest, dass du Zeit und Geld in Ersatz investieren musst, gehe bitte entsprechend vorsichtig damit um.

Der Lautsprecher ist nicht fest verklebt, sondern nur über zwei kleine Magnete auf der Platine gehalten und lässt sich dadurch abnehmen und wieder aufstecken. Falls er sich beim Transport verschoben hat, setze ihn wieder mittig auf die beiden Magnete auf, sodass er konzentrisch zur aufgedruckten Markierung sitzt. Zieh dabei nicht an den beiden dünnen Anschlusskabeln, sie sind die einzige elektrische Verbindung zum Lautsprecher und können sich sonst lösen oder abreißen.

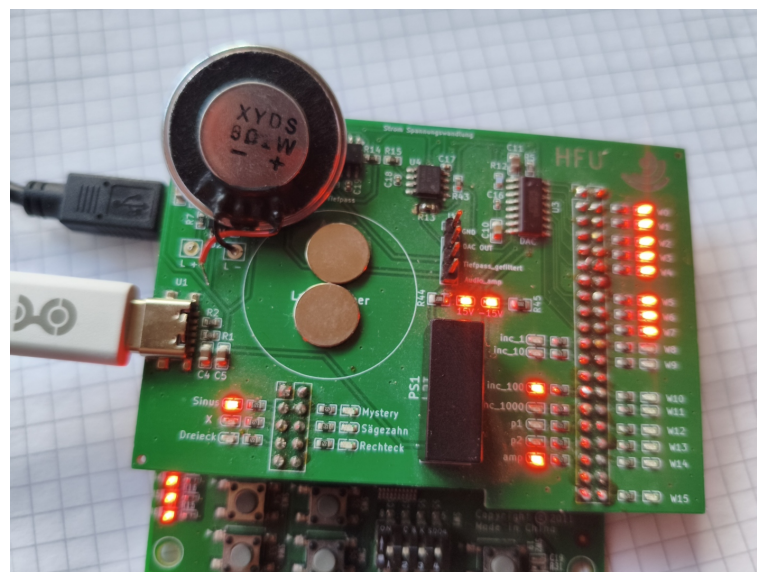


Abbildung 2.1: Lautsprecher abgenommen, darunter die beiden Magnete zur Befestigung

2.1 Stromversorgung und Anschluss

Für den Betrieb werden zwei USB-Kabel benötigt, die beide angeschlossen sein müssen.

- **USB Mini-B** (am FPGA-Board): wird mit dem Computer verbunden. Dieser Anschluss versorgt das FPGA-Board mit Strom und wird außerdem zum Programmieren des FPGA verwendet.
- **USB-C** (oben auf der Erweiterungsplatine): versorgt ausschließlich die Erweiterungsplatine mit Strom.

Ob die Spannungsversorgung der Erweiterungsplatine korrekt funktioniert, lässt sich direkt an den beiden LEDs mit der Beschriftung +15V und -15V ablesen. Leuchten beide, liegt die Versorgungsspannung an und die Platine ist betriebsbereit.

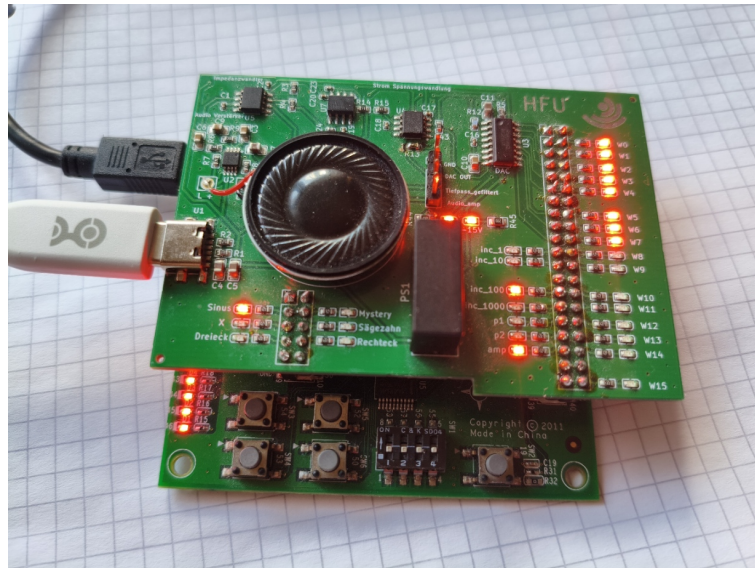


Abbildung 2.2: Aufgestecktes Board mit angeschlossenem USB Mini-B, die LED -15V zeigt eine korrekte Spannungsversorgung an

2.2 Signalabnahme

Zur Abnahme der Signale, etwa zum Anschließen eines Oszilloskops, befindet sich auf der Erweiterungsplatine eine Stiftleiste mit den Beschriftungen GND, DAC OUT, Tiefpass gefiltert und Audio_amp. Zum Abgreifen der einzelnen Signale werden Steckerleisten-Kabel (Jumper-Kabel) verwendet, die einfach auf die entsprechenden Stifte aufgesteckt werden, wie in Abbildung 2.3 zu sehen.

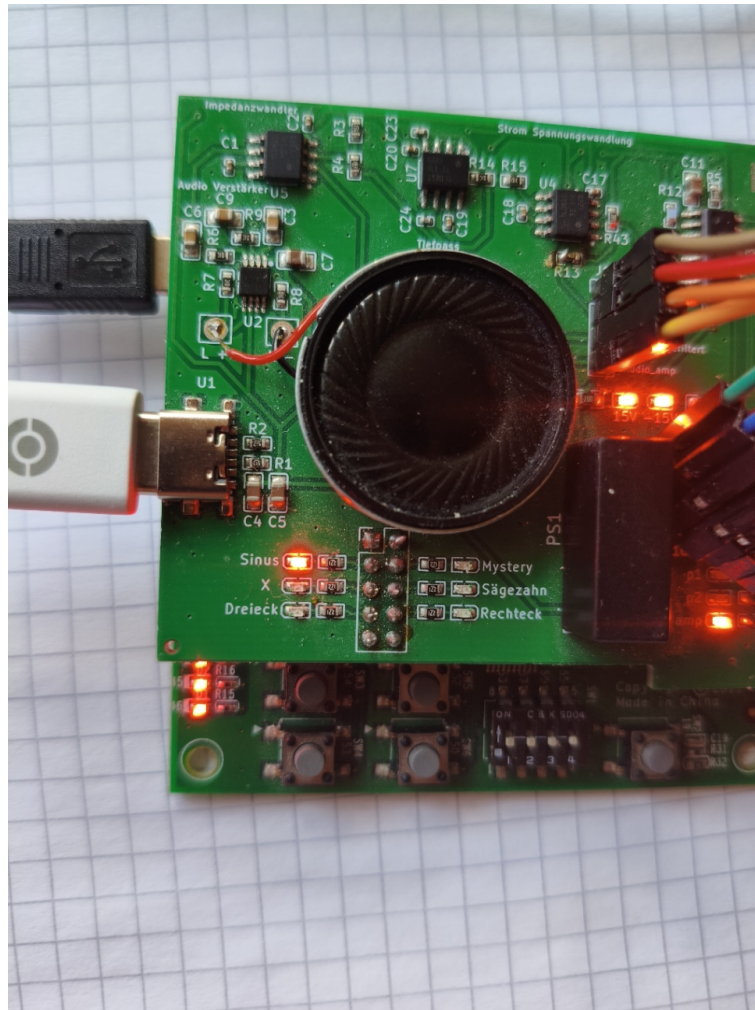


Abbildung 2.3: Signalabnahme über Steckerleisten-Kabel an der Stiftleiste

Um nachzuvollziehen, welche Station das Signal zwischen diesen vier Abgriffpunkten durchläuft, lohnt sich ein Blick in den beigefügten Schaltplan im Anhang. Er zeigt den vollständigen Signalpfad vom DAC-Ausgang über Tiefpassfilter und Audioverstärker bis zum Lautsprecher.

2.3 Anzeige LEDs

Um dir jederzeit mitzuteilen, was der Signalgenerator gerade tut, sind auf der Erweiterungsplatine mehrere LED Gruppen verteilt. Jede Gruppe hat eine eigene Funktion und zeigt dir einen bestimmten Zustand des Systems an.

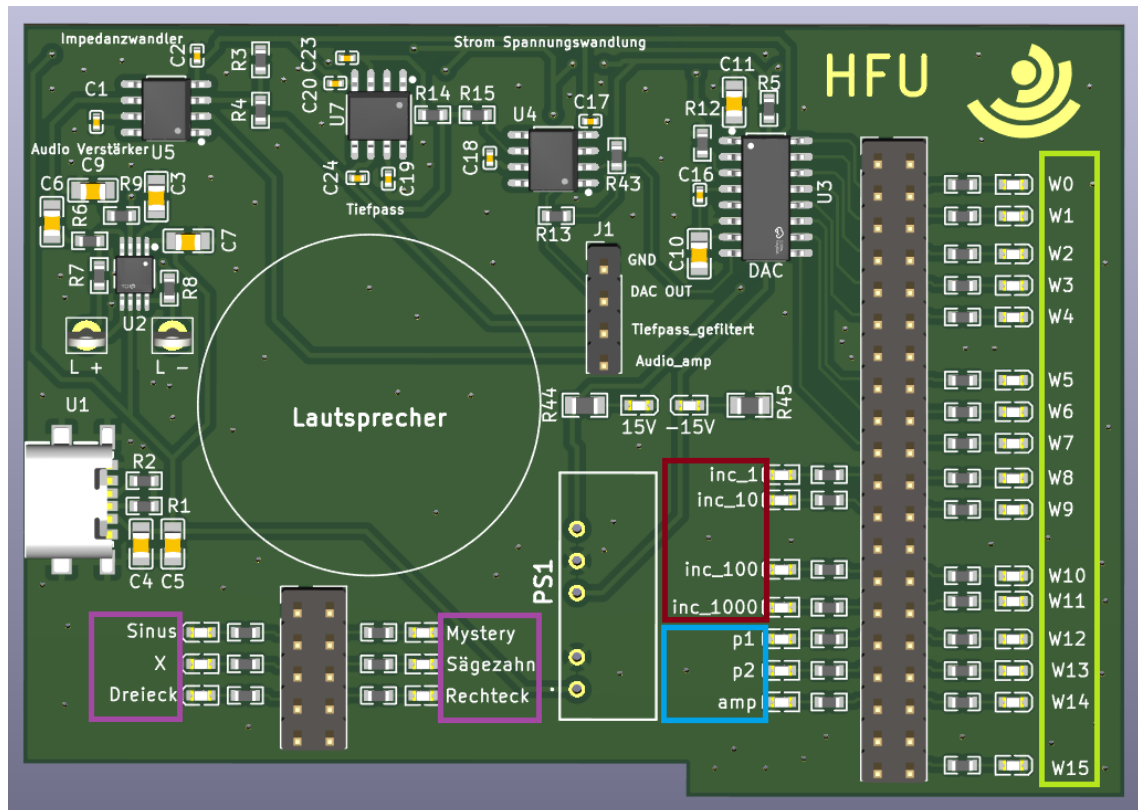


Abbildung 2.4: Signalgenerator Platine

- **Aktueller Parameterwert (grün)** Rechts im Bild befindet sich eine vertikale Reihe (grün) LEDs. Sie zeigen den *numerischen Wert* des aktuell ausgewählten Parameters (Amplitude, P1 oder P2) in binärer Form an. Wenn du also z. B. die Amplitude veränderst, kannst du hier direkt sehen, welchen Wert sie gerade hat.
- **Aktiver Parameter (blau)** Die blau markierten LEDs zeigen dir, *welcher Parameter* momentan ausgewählt ist. Immer genau eine der drei LEDs leuchtet.
 - LED 0: P1 (Feinfrequenz)
 - LED 1: P2 (Taktteiler)
 - LED 2: Amplitude
- **Schrittweite / Increment (rot)** Die rot markierten LEDs zeigen an, um welchen Betrag der aktuelle Parameter bei jedem Tastendruck erhöht oder verringert wird. Auch hier leuchtet immer genau eine LED, die zugehörige Schrittweite steht direkt daneben auf der Platine aufgedruckt. Diese Anzeige ist besonders hilfreich, wenn du schnell große Werte ändern möchtest.
- **Aktive Wellenform (lila)** Die lila markierten LEDs zeigen dir, welche Wellenform aktuell ausgegeben wird. Auch hier leuchtet immer genau eine LED, in der Anordnung aus Abbildung 2.4.
 - Sinus
 - X (deine Funktion)

- Dreieck
- Mystery
- Sägezahn
- Rechteck

Alle LEDs aktualisieren sich in Echtzeit und geben dir damit eine direkte Rückmeldung über den Zustand des Systems und die Wirkung deiner Eingaben.

2.4 Bedienung

Die in Abbildung 2.5 dargestellten Tasten bilden die komplette Benutzeroberfläche des digitalen Signalgenerators. Über fünf Taster können Wellenform, Parameter und Parameterwerte direkt am FPGA eingestellt werden.

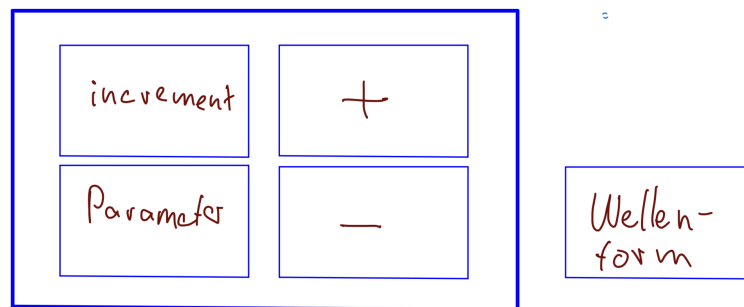


Abbildung 2.5: Skizze Tastenbelegung

Vier der fünf Taster sind als Block gruppiert: `btn_param` (SW3) unten links, `btn_inc` (SW4) oben links, `btn_plus` (SW5) oben rechts und `btn_minus` (SW6) unten rechts. Der Taster zur Wellenformauswahl, `btn_mode` (SW2), sitzt separat davon.

- `btn_mode` (SW2)
- `btn_param` (SW3)
- `btn_inc` (SW4)
- `btn_plus` (SW5)
- `btn_minus` (SW6)

Im Folgenden sind alle Tasten und ihre Funktion detailliert beschrieben.

Tastenbeschreibung

- **SW2 – Wellenform umschalten** (`btn_mode`) Durch jeden Tastendruck wird zur nächsten Wellenform weitergeschaltet. Die Reihenfolge entspricht der LED-Codierung:
 - Sinus
 - X (Reserve)
 - Dreieck
 - Mystery
 - Sägezahn
 - Rechteck

Die aktuell aktive Wellenform wird über `led_Modus` angezeigt.

- **SW3 – Parameterwahl** (`btn_param`) Mit diesem Taster wählst du aus, welcher Parameter verändert werden soll. Die Auswahl läuft zyklisch durch:
 - **P1** – Feinfrequenz / DDS-Inkrement
 - **P2** – globaler Taktteiler
 - **AMP** – Amplitude (0–255)

Der aktive Parameter wird über `led_param` (1-Hot) angezeigt.

- **SW4 – Schrittweite umschalten** (`btn_inc`) Dieser Taster bestimmt, wie stark ein Parameter pro Tastendruck verändert wird. Die Schrittweiten sind:
 - 1
 - 10
 - 100
 - 1000

Die aktive Schrittweite wird über `led_inc` angezeigt.

- **SW5 – Wert erhöhen** (`btn_plus`) Erhöht den aktuell ausgewählten Parameter um die eingestellte Schrittweite. Der neue Wert wird binär über `led_W` ausgegeben.
- **SW6 – Wert verringern** (`btn_minus`) Verringert den aktuell ausgewählten Parameter um die eingestellte Schrittweite. Auch hier zeigt `led_W` den aktuellen Wert an.

Zusammenspiel der Bedienung

Die Bedienung folgt einem einfachen Prinzip:

1. **Mit SW3 wählst du den Parameter** (P1, P2 oder AMP).
2. **Mit SW4 stellst du die Schrittweite ein** (1, 10, 100, 1000).
3. **Mit SW5 und SW6 erhöhst oder verringerst du den Wert.**

4. Mit SW2 wechselst du die Wellenform.

Alle Änderungen wirken sich sofort auf das Ausgangssignal `analog_out` aus, das über den DAC ausgegeben und am Oszilloskop sichtbar wird.

3 Funktionsweise

Nachdem Aufbau und Bedienung der Platine geklärt sind, geht es in diesem Kapitel um die interne Funktionsweise, wie die einzelnen Module zusammenspielen und nach welchem Prinzip jedes von ihnen sein Signal erzeugt. Dieses Verständnis erleichtert später den Einstieg in den Code der einzelnen Module und zeigt, an welcher Stelle du selbst weiterbauen musst.

3.1 Architektur im Überblick

Der Signalgenerator besteht aus einer Handvoll Modulen, die alle vom `TopLevel` aus miteinander verdrahtet werden.

- **Steuerung** wertet die fünf Taster aus und verwaltet, welcher Modus, welcher Parameter und welcher Wert gerade aktiv ist.
- **Frequenzteiler** sorgt dafür, dass alle Wellenformgeneratoren im richtigen Tempo laufen (dazu gleich mehr).
- Vier **Wellenformgeneratoren**, nämlich Rechteck, Dreieck, Sägezahn und Sinus, laufen die ganze Zeit parallel und rechnen jeweils ihr eigenes Signal aus.
- Ein **Multiplexer** pickt sich am Ende genau die Wellenform heraus, die gerade eingestellt ist, und legt sie auf den Ausgang.
- Der Ausgang geht 8 Bit breit, ein Pin pro Bit, auf den DAC und erzeugt dort die analoge Spannung, die du am Oszilloskop siehst.

Wichtig dabei, alle vier Generatoren laufen fortlaufend, unabhängig davon, welcher Modus gerade angezeigt wird. Es wird also nicht zwischen ihnen „umgeschaltet“ im Sinne von an/aus, der Multiplexer entscheidet nur, wessen Ausgabe gerade nach außen durchgereicht wird. Das hält die Schaltung einfach, weil kein Generator wissen muss, ob er gerade „dran“ ist oder nicht.

3.2 Ein Takt, viele Geschwindigkeiten

Wie lassen sich aus einem einzigen, festen 50 MHz-Systemtakt unterschiedlich schnelle Ausgabefrequenzen erzeugen? Dafür gibt es grundsätzlich zwei Möglichkeiten, entweder

man erzeugt sich selbst einen langsameren Takt, zum Beispiel aus einem Bit eines Zählers, oder man lässt alle Register am echten Systemtakt und steuert stattdessen über ein Enable-Signal, wann ein neuer Wert übernommen wird.

Ein selbstgebauter, langsamerer Takt kann auf Dauer zu Problemen führen, die sich erst auf der echten Hardware zeigen. Die Variante mit Enable-Signal umgeht das, kostet dafür aber mehr Leistung, die Taktleitung läuft im ganzen Design durchgehend mit 50 MHz, auch in den Teilen der Schaltung, die eigentlich viel langsamer arbeiten könnten. In diesem Projekt wird die Enable-Variante verwendet, *alle* Register bleiben am echten Systemtakt, bekommen aber zusätzlich ein ganz normales Signal spendiert, das nur gelegentlich für einen Takt lang auf 1 geht, ein sogenanntes *Enable-Signal*. Jedes Register prüft bei jeder Taktflanke, ob das Enable-Signal gerade aktiv ist, und übernimmt einen neuen Wert nur dann. Ist es das nicht, bleibt der alte Wert einfach stehen, obwohl der Takt weiterläuft.

Im Projekt heißt dieses Signal `tick_en` und wird vom Modul `frequenzteiler` erzeugt. Ein Zähler läuft im Systemtakt hoch und setzt `tick_en` für genau einen Takt auf 1, sobald er den Wert P2 erreicht hat. Je größer P2, desto seltener wird `tick_en` aktiv, desto langsamer laufen alle Generatoren, und desto tiefer wird der erzeugte Ton. P2 ist also der grobe, gemeinsame Geschwindigkeitsregler für das ganze System.

Wichtig dabei, der Zähler im `frequenzteiler` ist einstufig, keine Kaskade mehrerer Zähler hintereinander. Deshalb lässt sich `tick_en` auch einfach als Teilung beschreiben, `tick_en` hat die Frequenz $\frac{50 \text{ MHz}}{P2}$. Das brauchst du später bei der Frequenzberechnung.

3.3 Wellenformerzeugung

Alle vier Generatoren bekommen denselben Systemtakt und dasselbe `tick_en`, rechnen ihr Signal aber nach unterschiedlichem Prinzip aus.

3.3.1 Rechteck

Ein Zähler läuft mit jedem `tick_en`-Puls hoch. Erreicht er den eingestellten Wert P1, kippt der Ausgang zwischen einem oberen und einem unteren Wert um und der Zähler beginnt von vorn. Weil High- und Low-Phase über denselben Zähler und denselben Vergleichswert laufen, ist das Tastverhältnis unabhängig von P1 immer exakt 50 %.

3.3.2 Dreieck

Hier zählt ein Wert schrittweise hoch bis zu einer oberen Grenze und dann wieder runter bis zur unteren Grenze, endlos im Wechsel. P1 legt fest, wie viele `tick_en`-Pulse ein einzelner Schritt dauert, je größer P1, desto langsamer und „gestreckter“ wird die Dreieckschwingung.

3.3.3 Sägezahn

Fast wie das Dreieck, nur ohne das Runterzählen. Der Wert steigt schrittweise bis zur oberen Grenze und springt dann hart zurück auf die untere Grenze. Das ergibt die charak-

teristische Sägezahnform statt eines symmetrischen Dreiecks.

3.3.4 Sinus (DDS)

Der Sinus ist der aufwendigste Generator, weil ein echter Sinus sich nicht einfach hoch- und runterzählen lässt. Stattdessen kommt das DDS-Prinzip (Direct Digital Synthesis) zum Einsatz.

1. Ein *Phasenakkumulator* zählt mit jedem `tick_en`-Puls um einen Schritt weiter, dessen Größe `P1` bestimmt. Ein voller Überlauf dieses Zählers entspricht genau einer vollen 360°-Umdrehung. Je größer `P1`, desto schneller dreht sich die Phase, desto höher die Frequenz.
2. Ein fertiger IP-Baustein (der `SIN`-Kern) übersetzt die aktuelle Phase über eine hinterlegte Sinustabelle in einen Rohwert.
3. Eine kleine Rechenpipeline skaliert diesen Rohwert auf die eingestellte Amplitude und verschiebt ihn von einem vorzeichenbehafteten Bereich in den Bereich 0–255, den der DAC erwartet.

3.4 Tasterentprellung

Mechanische Taster „prellen“, beim Drücken wechseln sie für ein paar Millisekunden mehrfach schnell zwischen 0 und 1, statt sauber einmal umzuschalten. Würde man das direkt auswerten, würde ein einziger Tastendruck als mehrere Tastendrucke gezählt. Das Modul `debounce` übernimmt deshalb einen neuen Tasterwert erst, wenn er eine festgelegte Zeit lang stabil ansteht, und synchronisiert das von außen kommende Signal zusätzlich über zwei Flipflops auf den Systemtakt. Erst danach wertet die `steuerung` den eigentlichen Tastendruck (die fallende Flanke) aus.

4 Aufgaben

Die folgenden Aufgaben bauen aufeinander auf. Zuerst geht es darum, den erklärten Code wirklich verstanden zu haben, danach folgt eine Rechenaufgabe zum DDS-Prinzip, und zum Schluss zwei offene, kreative Aufgaben, bei denen du selbst etwas entwirfst. Für die Rechenaufgabe findest du am Ende des Kapitels eine Lösung, für die kreativen Aufgaben gibt es bewusst keine Musterlösung.

4.1 Fehlende Codeteile rekonstruieren

In eurem Projektordner sind an einigen Stellen Codezeilen nicht mehr vorhanden, offenbar hat sich ein hungriger Wurm durch die Dateien gefressen. Die betroffenen Stellen sind mit - TODO markiert. Arbeite die Module in dieser Reihenfolge durch, sie bauen von einfach zu anspruchsvoll aufeinander auf. Nach jedem Schritt kompilieren und die betroffene Wellenform am Oszilloskop prüfen, erst dann weiter zum nächsten Modul.

1. frequenzteiler.vhd

Der Grundbaustein für alles Weitere, `tick_out` muss alle `divisor_in` Systemtakte für genau einen Takt auf '1' gehen. Dazu zählt `counter` bei jedem Takt hoch, bei Erreichen von `divisor_in - 1` wird `counter` zurückgesetzt und `tick_reg` für diesen einen Takt auf '1' gesetzt, sonst bleibt `tick_reg` auf '0'. Ohne dieses Modul bekommt keiner der Generatoren sein `tick_en`, es lohnt sich also, hier zuerst anzusetzen.

2. rect_gen.vhd

Der Rechteckgenerator ist der einfachste Fall, hier zählst du nur einen Zähler hoch und schaltest bei Erreichen einer Grenze um. Im process fehlt genau das, `counter` muss bei jedem `tick_en`-Puls um 1 erhöht werden, und sobald er `ticks_per_phase` erreicht, muss er auf 0 zurückgesetzt und `state` umgeschaltet (invertiert) werden. Wie `state` danach den Ausgang steuert, steht schon fertig unten im Code, das musst du nicht anfassen.

3. dreieck_gen.vhd

Fast derselbe Zählermechanismus wie eben, nur zählt hier zusätzlich `current_val` selbst hoch oder runter, gesteuert über `is_rising`. Es fehlt Folgendes, wenn die Schrittdauer (`tick_counter` gegen `ticks_limit`) erreicht ist, muss `current_val` je nach `is_rising`

um 1 erhöht oder verringert werden. Erreicht `current_val` dabei `max_val`, kippt `is_rising` auf '0' (jetzt fallend), erreicht es `min_val`, kippt es zurück auf '1' (jetzt steigend).

4. saegezahn_gen.vhd

Wieder derselbe Zählermechanismus, diesmal ohne Richtungswechsel, `current_val` zählt nur hoch. Es fehlt, bei Erreichen der Schrittdauer entweder `current_val` um 1 zu erhöhen, oder, falls `max_val` bereits erreicht ist, hart zurück auf `min_val` zu springen (kein Herunterzählen wie beim Dreieck).

5. Den SIN-Kern erzeugen

Für den nächsten Schritt, den Sinusgenerator, wird ein fertiger IP-Baustein für die Sinustabelle gebraucht. Die Datei `SIN.vhd` fehlt aktuell komplett und muss über IPexpress neu erzeugt werden, sonst lässt sich ab hier nichts mehr kompilieren (auch der Mystery-Modus aus Aufgabe 4.4 braucht denselben Kern).

Öffne in Diamond den IP-Katalog und wähle unter *Arithmetic_Modules* das Modul *Sin-Cos_Table*. Trage im ersten Dialog als Dateiname `SIN` ein und wähle als Module Output VHDL (Abbildung 4.1).

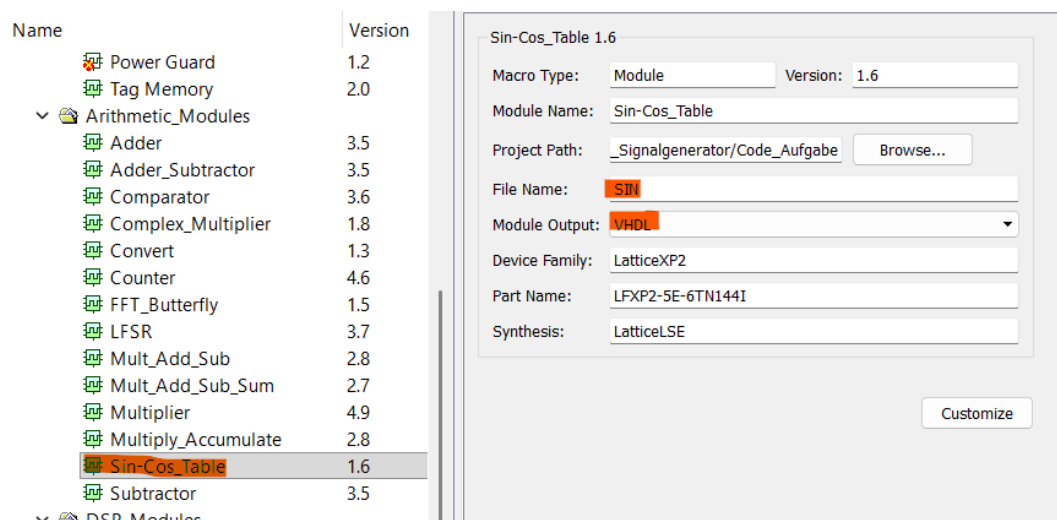


Abbildung 4.1: IPexpress-Katalog, Modulauswahl Sin-Cos_Table

Im Konfigurationsdialog müssen folgende Einstellungen gesetzt werden, damit der erzeugte Kern zum restlichen Code passt (Abbildung 4.2).

- Input Theta Bit Width, 10
- Output Mode, Sin
- „Use Tables for Quarter-wave Only“ **nicht** angehakt
- Enable Input Registers und Enable Output Registers bleiben angehakt

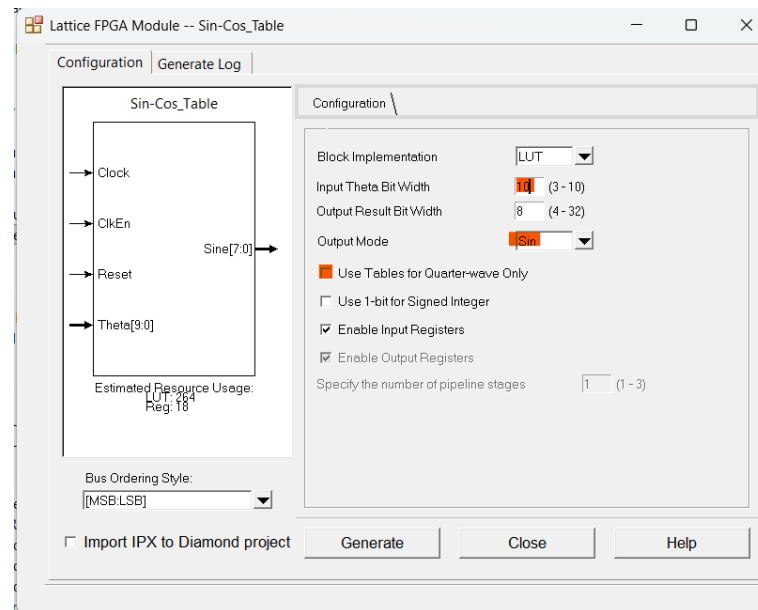


Abbildung 4.2: IPexpress-Konfiguration, korrekte Einstellungen

Mit *Generate* wird `SIN.vhd` neu erzeugt.

6. sinus_gen.vhd

Der eigentliche DDS-Kern aus Kapitel 3, hier fehlt gleich zweimal etwas. Erstens muss `phase_acc` bei jedem `tick_en`-Puls um `phase_inc` erhöht werden (auf die 10 Bit des Akkumulators resized). Zweitens, in der Skalierungspipeline nach dem SIN-Kern, muss `sine_raw` vorzeichenrichtig auf 16 Bit erweitert, mit `amp_in` multipliziert, um 8 Bit runterskaliert und danach um 128 verschoben werden, damit aus dem vorzeichenbehafteten Sinuswert (-128..127) ein für den DAC gültiger Wert (0..255) wird. Die genauen vier Schritte der Pipeline stehen als Kommentar im Code.

7. debounce.vhd

Hier fehlt die eigentliche Entprellungslogik. `counter` muss hochzählen, solange sich `sync1` vom zuletzt übernommenen `stable_val` unterscheidet. Erreicht `counter` dabei `DEBOUNCE_CYCLES`, wird `sync1` als neuer `stable_val` übernommen und `counter` zurückgesetzt. Stimmen `sync1` und `stable_val` dagegen schon überein (kein Prellen mehr erkannt), wird `counter` ebenfalls zurückgesetzt.

8. steuerung.vhd

Taster 5 (Wert erhöhen) ist bereits vollständig implementiert, schau ihn dir als Vorlage an. Es fehlt Taster 6 (Wert verringern), spiegelbildlich zu Taster 5, aber mit einer Besonderheit, `unsigned` kann nicht negativ werden. `p1_reg` und `p2_reg` müssen deshalb bei 1 stehen bleiben (nicht bei 0), `amp_reg` bei 0.

9. toplevel.vhd

Der letzte und wichtigste Schritt, der Multiplexer, der alle Generatoren zusammenführt. Erst wenn der steht, läuft das Gesamtsystem. Es fehlt eine `with sig_mode select`-Zuweisung auf `final_signal`, die je nach Modus die passende Wellenform durchreicht, "001" Sinus, "010" stumm (`x"00"`), "011" Dreieck, "100" Rechteck, "101" Sägezahn, "110" Mystery, alle anderen Werte ebenfalls stumm.

4.2 Frequenzberechnung beim Sinusgenerator

Der Sinusgenerator arbeitet nach dem DDS-Prinzip mit einem 10 Bit breiten Phasenakkumulator, ein voller Umlauf entspricht also 1024 Schritten. Der Phasenakkumulator wird bei jedem `tick_en`-Puls um `P1` erhöht, und `tick_en` selbst wird alle `P2` Systemtakte einmal aktiv, bei einem Systemtakt von 50 MHz.

1. Stelle eine Formel auf, die die Ausgabefrequenz f_{out} in Abhängigkeit von `P1`, `P2` und dem Systemtakt beschreibt.
2. Bestimme jeweils ein Wertepaar (`P1`, `P2`), mit dem der Sinusgenerator eine Ausgabefrequenz von etwa 100 Hz beziehungsweise etwa 1 kHz erzeugt. Das muss nicht exakt ausgerechnet sein, probier ruhig auch, bis es passt. Achte darauf, `P1` kleiner als 1024 zu wählen, größere Werte werden im Code auf 10 Bit gekürzt und ergeben dadurch eine andere Frequenz als erwartet.
3. Stelle deine Werte ein und überprüfe direkt am Oszilloskop, ob die Frequenz stimmt.

4.3 Eigene Wellenform auf Modus X

Modus 2 ist im Multiplexer des Toplevels aktuell auf stumm geschaltet und in der LED-Anzeige als „X (deine Funktion)“ reserviert. Genau dafür ist er da.

Suche dir eine mathematische Funktion deiner Wahl aus, zum Beispiel eine Betragssinuskurve, eine Exponentialkurve, eine Kombination aus zwei der bestehenden Wellenformen oder etwas völlig anderes, und baue sie nach. Implementiere sie als eigenständiges Modul nach demselben Muster wie `rect_gen`, `dreieck_gen`, `saegezahn_gen` oder `sinus_gen`, und binde es im Toplevel anstelle des stummen Platzhalters bei Modus 2 ein.

4.4 Eigenen Ton für den Mystery-Modus

In `mystery.vhd` ist so gut wie nichts mehr übrig, der Wurm hatte hier besonderen Appetit. Deine Aufgabe, bau dir selbst etwas, das bei Modus 6 einen Ton ausgibt.

Orientiere dich dabei am Sinusgenerator aus Aufgabe 4.1, gleiches DDS-Prinzip, derselbe SIN-Kern. Der Unterschied ist, dass die Frequenz hier nicht über einen Taster kommt, sondern sich automatisch nach einer festen Zeit ändern soll. Für den Anfang reicht es, wenn

dabei irgendein fester Ton zu hören ist, das zeigt schon, dass Zeittaktung, Phasenakkumulator und SIN-Kern richtig zusammenspielen.

Bonus, eine echte Melodie

Statt nur eines einzelnen Tons, bau eine Abfolge mehrerer Noten, die tatsächlich wie eine kleine Melodie klingt. Welche Noten, in welcher Reihenfolge und mit welchem Tempo, darfst du dir gerne von einer KI deiner Wahl vorschlagen lassen.

4.5 Zusatzaufgabe, LEDs im Takt der Melodie blinken lassen

Im Toplevel gibt `mystery` bereits über den Port `note_dbg` die Nummer der gerade gespielten Note aus, aktuell aber unbenutzt. Deine Aufgabe, nutze dieses Signal, um in Modus 6 die Anzeige-LEDs (`led_param`, `led_inc`, `led_modus`, `led_w`) statt ihrer normalen Bedeutung ein sich veränderndes Muster anzeigen zu lassen, das bei jeder neuen Note automatisch weiterschaltet, zum Beispiel im Wechsel zwischen mehreren fest definierten Bitmustern.

Wichtig dabei, die normale Anzeige (Parameter, Schrittweite, Wellenform, Wert) muss in den anderen fünf Modi weiterhin ganz normal funktionieren, nur in Modus 6 soll stattdessen das Muster erscheinen.

4.6 Lösungen

Frequenzberechnung. Ein voller Umlauf des Phasenakkumulators entspricht 1024 Schritten. Pro `tick_en`-Puls wächst er um `P1`, und `tick_en` hat die Frequenz $\frac{50 \text{ MHz}}{P2}$. Damit ergibt sich

$$f_{\text{out}} = \frac{P1}{1024} \cdot \frac{50 \text{ MHz}}{P2}$$

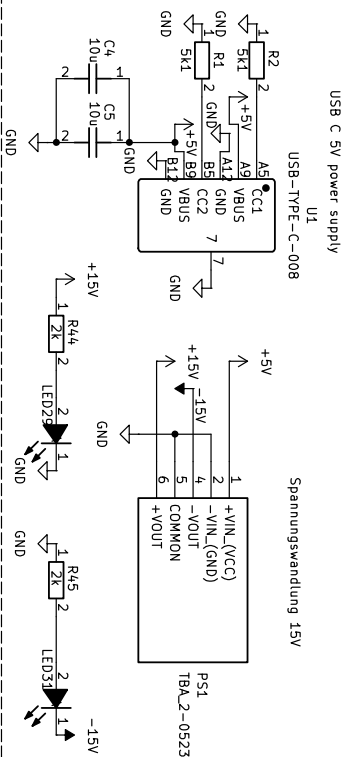
Für exakt 100 Hz, $P1 = 32$, $P2 = 15625$. Für exakt 1 kHz, $P1 = 64$, $P2 = 3125$. Es gibt jeweils noch weitere gültige Kombinationen, diese beiden sind nur ein Beispiel.

Abbildungsverzeichnis

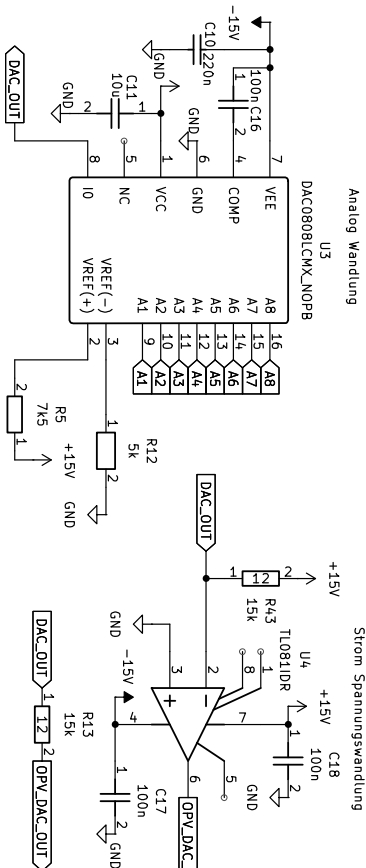
2.1	Lautsprecher abgenommen, darunter die beiden Magnete zur Befestigung .	2
2.2	Aufgestecktes Board mit angeschlossenem USB Mini-B, die LED -15V zeigt eine korrekte Spannungsversorgung an	3
2.3	Signalabnahme über Steckerleisten-Kabel an der Stiftleiste	4
2.4	Signalgenerator Platine	5
2.5	Skizze Tastenbelegung	6
4.1	IPexpress-Katalog, Modulauswahl Sin-Cos_Table	13
4.2	IPexpress-Konfiguration, korrekte Einstellungen	14

5 Anhang

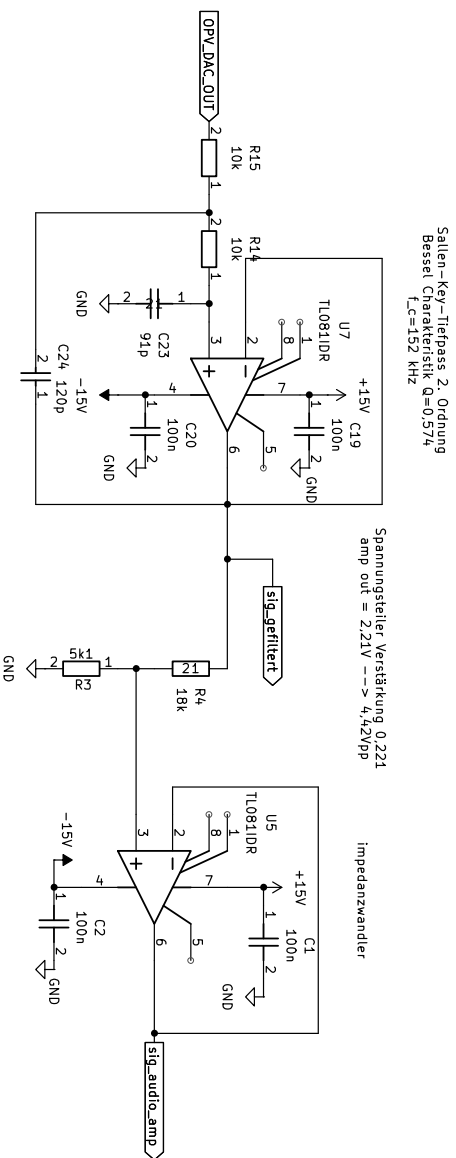
Spannungsversorgung



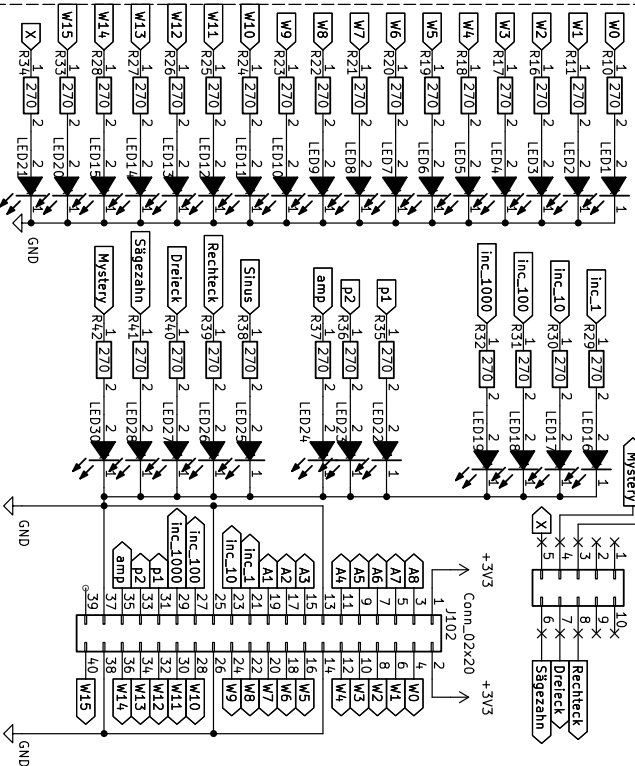
DA Wandler



Audio Signallaufbereitung



FPGA – Board



HFU

File: Zusatzplatine.kicad_sch

Title: Signalgenerator

Size

A4	
----	--

Date: 2026-03-29

KICad E.D.A. 9.0.4

100

Id: 1/1